

Universidade Federal do Amazonas
Instituto de Computação ICC062 - Arquitetura de Computadores 2024/2
Aluno: Nycksandro Lima dos Santos
Matrícula: 22351228

Trabalho Prático 07: Script de Detecção de Bordas usando GPU e Numba

1) Explicação dos trechos de código mostrando o uso do Numba para aceleração em GPU.

Bibliotecas utilizadas:

```
[1] #ICC062 - Arquitetura de Computadores 2024/2
#Trabalho Prático 07: Script de Detecção de Bordas usando GPU e Numba
##Aluno: Nycksandro Lima dos Santos
#Matrícula: 22351228

import time # uso para marcar o tempo
import numpy as np
import matplotlib.pyplot as plt
from numba import cuda
from PIL import Image #uso para imagem
import math #uso para arredondar
from google.colab import files # uso para fazer upload da imagem
```

Figura 01- Bibliotecas que utilizo para realização do trabalho prático.

Implementação da Função de Convolução:

```
def convulcaoMatriz(matriz_copia, matriz_resultante, mascaraGx, mascaraGy): # i = índice linha, j = índice coluna
    i,j = cuda.grid(2) # Define i e j como índices globais que percorrem a matriz

    somatorio_convulcaoGx = 0 # Crio uma variável acumuladora para Gx
    somatorio_convulcaoGy = 0 # Crio uma variável acumuladora para Gy

    if(i < matriz_resultante.shape[0] and j < matriz_resultante.shape[1]):
        #Parte de Gx
        somatorio_convulcaoGx += matriz_copia[i,j] * mascaraGx[0][0]
        somatorio_convulcaoGx += matriz_copia[i,j+1] * mascaraGx[0][1]
        somatorio_convulcaoGx += matriz_copia[i,j+2] * mascaraGx[0][2]
        somatorio_convulcaoGx += matriz_copia[i+1,j] * mascaraGx[1][0]
        somatorio_convulcaoGx += matriz_copia[i+1,j+1] * mascaraGx[1][1]
        somatorio_convulcaoGx += matriz_copia[i+1,j+2] * mascaraGx[1][2]
        somatorio_convulcaoGx += matriz_copia[i+2,j] * mascaraGx[2][0]
        somatorio_convulcaoGx += matriz_copia[i+2,j+1] * mascaraGx[2][1]
        somatorio_convulcaoGx += matriz_copia[i+2,j+2] * mascaraGx[2][2]

        #Parte Gy
        somatorio_convulcaoGy += matriz_copia[i,j] * mascaraGy[0][0]
        somatorio_convulcaoGy += matriz_copia[i,j+1] * mascaraGy[0][1]
        somatorio_convulcaoGy += matriz_copia[i,j+2] * mascaraGy[0][2]
        somatorio_convulcaoGy += matriz_copia[i+1,j] * mascaraGy[1][0]
        somatorio_convulcaoGy += matriz_copia[i+1,j+1] * mascaraGy[1][1]
        somatorio_convulcaoGy += matriz_copia[i+1,j+2] * mascaraGy[1][2]
        somatorio_convulcaoGy += matriz_copia[i+2,j] * mascaraGy[2][0]
        somatorio_convulcaoGy += matriz_copia[i+2,j+1] * mascaraGy[2][1]
        somatorio_convulcaoGy += matriz_copia[i+2,j+2] * mascaraGy[2][2]

    #Não usei numpy para fazer a operação de raiz quadrada pois sqrt do numpy não é compatível com numba
    matriz_resultante[i,j] = ((somatorio_convulcaoGx**2) + (somatorio_convulcaoGy**2))**0.5 # Realiza o calculo da magnitude e coloca na posição do pixel correspondente
```

Figura 02- A função de convolução que implementei recebe a matriz de entrada (matriz_copia), uma matriz resultante, a máscara para Gx, a máscara para Gy e realiza convolução das duas máscaras, também fazendo a soma dos quadrados de Gx e Gy, tirando a raiz quadrada e colocando o resultado no pixel correspondente ao centro da máscara.

Implementação da Função de aplicação do Filtro:

```
def algoritmoSobel(matriz_entrada, tam_mascara = 3, stride_mascara = 1): # Algoritmo de Sobel que recebe uma matriz de entrada e uma matriz de saída, utilizando como tamanho da mascara = 3 e o stride da =
    if(tam_mascara == 3 and stride_mascara == 1): # Se a função receber os parametros certos (tamanho da mascara = 3 e stride da mascara = 1)

        #Defino as mascaras 3x3 já invertidas
        mascaraGx = np.array([[[-1, 0, 1],[-2, 0, 2],[1, 0, 1]], dtype = np.float32) # Mascara Gx já invertida
        mascaraGy = np.array([[[-1, 2, -1],[0, 0, 0],[1, 2, 1]], dtype = np.float32) # Mascara Gy já invertida

        matriz_copia = np.pad(matriz_entrada,[1,1], "constant", constant_values = (0,0)).astype(dtype = np.float32) # Utilizando a função do numpy "pad" para realizar o padding da imagem

        matriz_resultante = np.zeros((len(matriz_copia[0])-2, len(matriz_copia)-2),dtype= np.float32) # Criando uma matriz cheia de zeros para servir de matriz resultante

        #Realizando o calculo para descobrir quantas mascaras serão necessária:
        largura_necessaria = math.ceil((len(matriz_copia[0]) - tam_mascara)/stride_mascara) + 1 # Aplicando a fórmula para calcular quantas máscaras serão necessárias
        altura_necessaria = math.ceil((len(matriz_copia) - tam_mascara)/stride_mascara) + 1 # Aplicando a fórmula para calcular quantas máscaras serão necessárias

        #Calculando a quantidade de blocos necessária:
        #So cada bloco da GPU está associado à execução de uma máscara, então o número total de blocos será o mesmo do número total de máscaras. (Na minha implementação estou usando 1 thread que aplica nas d
        #O número total de máscaras (contando as duas como uma) = largura_necessaria * altura_necessaria
        #Logo: a grid terá a dimensão de (largura_necessaria, altura_necessaria)
        #cada bloco ficará responsável pela execução de 1 thread

        blocos_grid = (largura_necessaria, altura_necessaria) # Definindo uma tupla que receberá a dimensão da quantidade de blocos (grid)
        threads = (1,1) # 1 thread apenas

        #Copiando as variáveis necessárias para aplicação da função para a GPU e iniciando o timer de marcação (copiar a variável para GPU também conta para a contagem)

        inicio_tempo = time.time() #Início da marcação do tempo

        matriz_copia_gpu = cuda.to_device(matriz_copia) #copia para a memoria da gpu
        matriz_resultante_gpu = cuda.to_device(matriz_resultante) # copia para a memoria da gpu
        mascaraGx_gpu = cuda.to_device(mascaraGx) #copia para a memoria da gpu
        mascaraGy_gpu = cuda.to_device(mascaraGy) #copia para a memoria da gpu

        #Aplicação da função paralelizada
```

Figura 03- O filtro de Sobel que implementei recebe a matriz de entrada, copia e converte para um matriz numpy, realiza o padding, define as dimensões do bloco de grid e a quantidade de threads a ser utilizada com base nas fórmulas da especificação do trabalho prático, copia as variáveis necessárias para a memória da GPU aplica todas as convoluções de forma paralela, calcula e mostra o tempo decorrido para a aplicação dessas convoluções e retorna essa matriz numpy com o filtro aplicado.

```
#Aplicação da função paralelizada

convulcaoMatriz(blocos_grid, threads)(matriz_copia_gpu, matriz_resultante_gpu, mascaraGx_gpu, mascaraGy_gpu) # Aplica a convolução de forma paralelizada nas duas máscaras, calculando Gx e Gy, soma os quadrados e tira a raíz, onde

matriz_resultante = matriz_resultante_gpu.copy_to_host() #copia da memoria da gpu para variavel

fin_tempo = time.time() #Fim da marcação do tempo

tempo_decorrido = fin_tempo - inicio_tempo #Calculando quanto tempo se passou

print(f"Tempo de execução: {tempo_decorrido} segundos")
return matriz_resultante #retorna a matriz já com Sobel aplicada

else: #Mensagem de erro caso esteja no formato errado
    print("O algoritmo só funciona para tamanho da mascara = 3 e stride = 1")
```

Figura 04- Continuação da função do filtro de Sobel que implementei.

Conversão da matriz numpy gerada para imagem RGB:

```
#Célula para colocar as imagens

imagem_upload = files.upload() # Carrego uma imagem
imagem_upload = list(imagem_upload.keys())[-1] # pego o elemento inserido

imagem = Image.open(imagem_upload) # Aqui dentro coloca o caminho da imagem ou daa imagem carregada

if(imagem.mode != "L"): #Verifico se a imagem não é de escala de cinzas (2D), se for eu converto para escalas de cinza
    imagem = imagem.convert("L") # Convertendo para escala de cinza

imagem = imagem.resize((600,600)) # Utiliza o resize escolhendo a dimensão da imagem

imagem_pos_filtro = algoritmoSobel(imagem) # Aplica o filtro de Sobel na imagem e obtém a matriz numpy dessa imagem

imagem_pos_filtro = Image.fromarray(imagem_pos_filtro) # Transforma a matriz numpy em uma imagem RGB

plt.axis("off") # Tirando os eixos da imagem (vou plotar a imagem)
plt.imshow(imagem_pos_filtro) # Mostrando a imagem

print(f"As dimensões das imagens são: {imagem_pos_filtro.size}") # Printo as dimensões da imagem
```

Figura 05- Célula que utilizo para fazer upload da imagem (pega a mais recente), verifico se a imagem está na escala de cinza, se não estiver converto para escala de cinza, e também defino o tamanho da imagem através da função resize(), aplicando o filtro de Sobel que implementei, após isso converto a matriz numpy resultante para

uma imagem RGB (padrão do `fromarray()`), e ploto a imagem com a biblioteca `matplotlib`, mostrando as dimensões atuais da imagem gerada.

2) Um gráfico de barras de erro relacionado a 15 execuções para cada tamanho de imagem.

A transferência das variáveis para a memória da GPU e a transferência da memória da GPU para variável também é considerada para a contagem de tempo de execução da função na GPU.

Utilizei a função `time()` da biblioteca `time` logo antes da transferência das variáveis para a memória da GPU e utilizei a função `time()` novamente após a transferência da matriz da memória para a variável e realizei a diferença desses dois valores, assim obtendo o tempo decorrido na aplicação do filtro.

Para cada dimensão de imagem (600x600, 1500x1500 e 2100x2100) e para cada número de threads (1,4,9,16,25,36), eu realizei 15 execuções do filtro Sobel e tirei a média aritmética desses tempos (em milissegundos), também calculando o seu desvio padrão.

A imagem utilizada para essas execuções foi a fornecida no anexo do colabweb.

Tabela:

Número de Threads	600x600	1500x1500	2100x2100
(1,1)	49.884	72.144	133.356
(2,2)	35.077	37.704	62.665
(3,3)	47.354	27.742	51.353
(4,4)	31.402	20.396	38.299
(5,5)	32.329	17.945	76.381
(6,6)	4.607	17.056	31.227

Tabela 01: Tabela que mostra detalhadamente a relação entre o número de threads e tamanho da imagem e o tempo de execução do filtro de Sobel na GPU (em milissegundos).

Como visto na tabela, o número de threads que apresentou o melhor desempenho foram de 16 e 36 threads.

Gráficos:

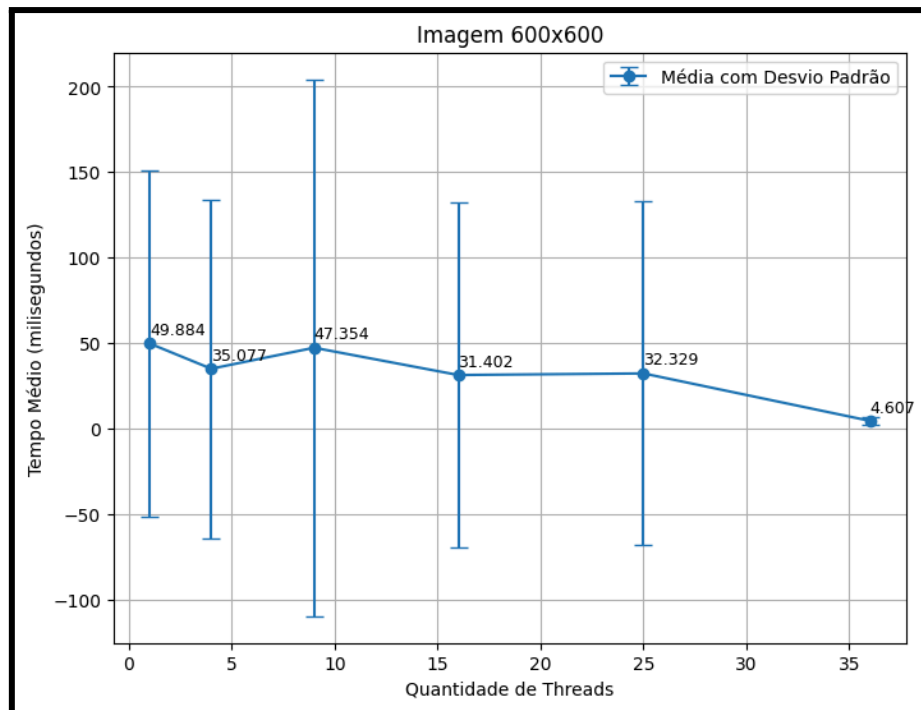


Figura 06- Gráfico que mostra a relação entre o número de threads (1,4,9,16,25,36) por bloco e o tempo decorrido na execução das convoluções (em milissegundos) em uma imagem 600x600.

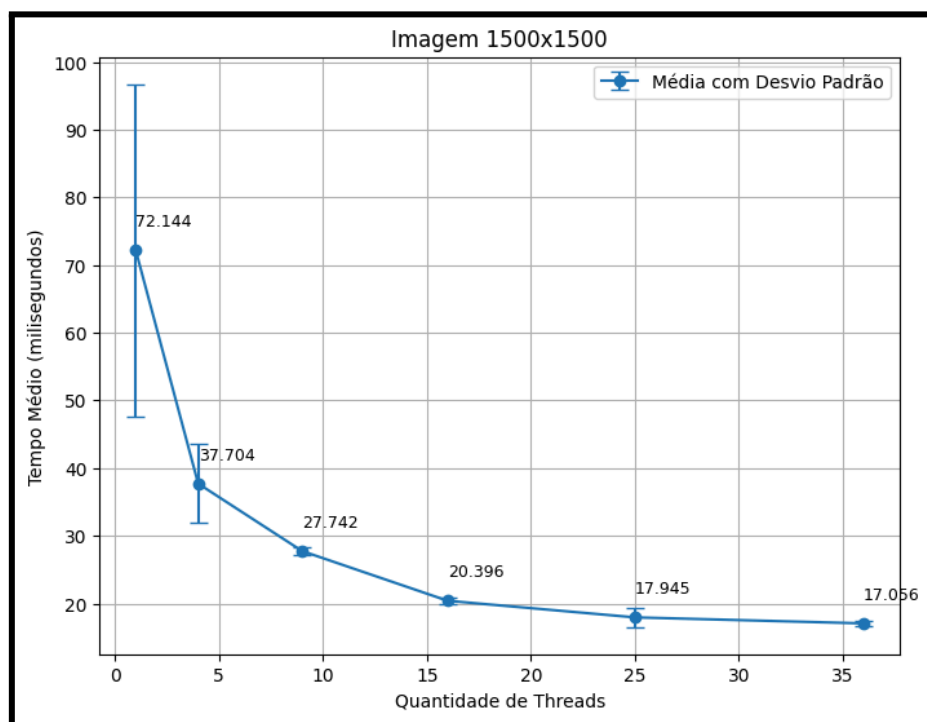


Figura 07- Gráfico que mostra a relação entre o número de threads (1,4,9,16,25,36) por bloco e o tempo decorrido na execução das convoluções (em milissegundos) em uma imagem 1500x1500.

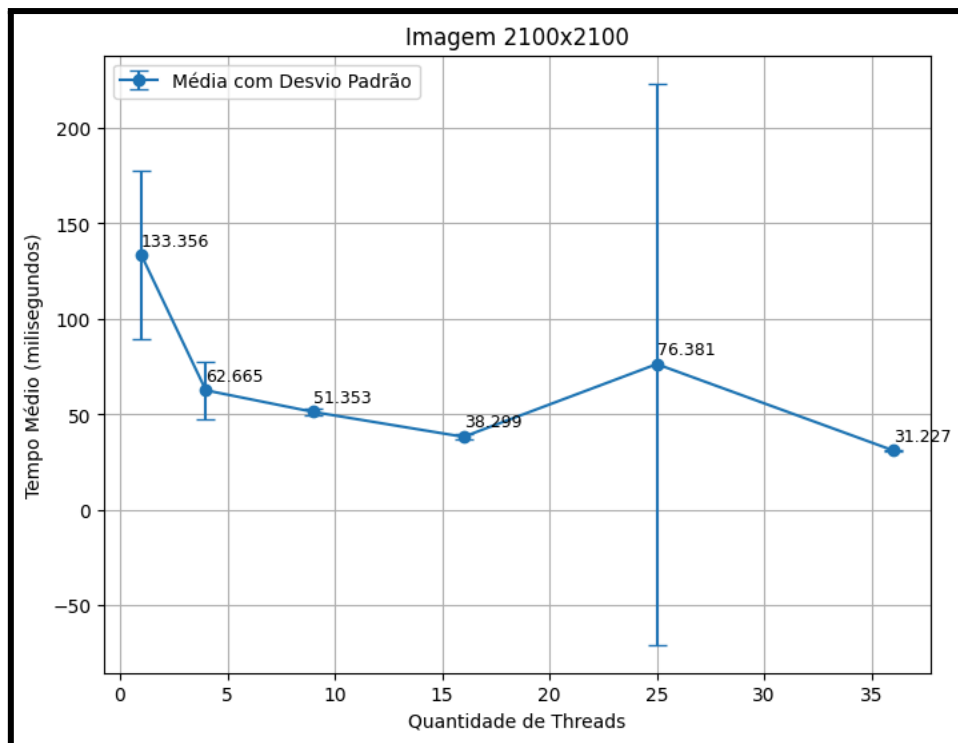


Figura 08- Gráfico que mostra a relação entre o número de threads (1,4,9,16,25,36) por bloco e o tempo decorrido na execução das convoluções (em milissegundos) em uma imagem 2100x2100.

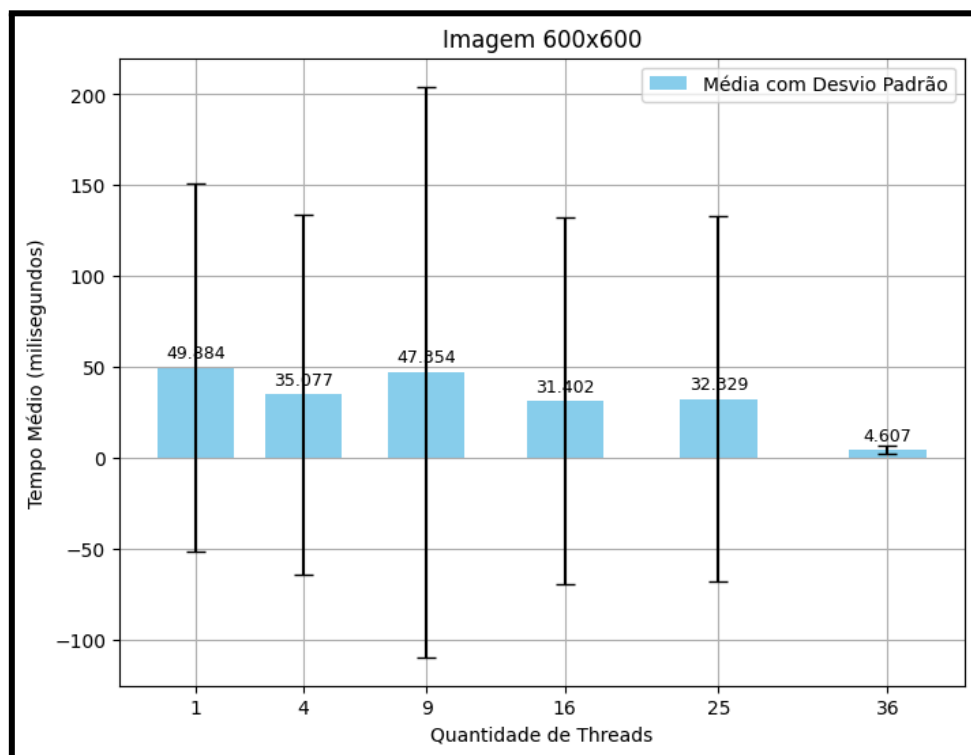


Figura 09- O mesmo gráfico que a Figura 06 só que em barras.

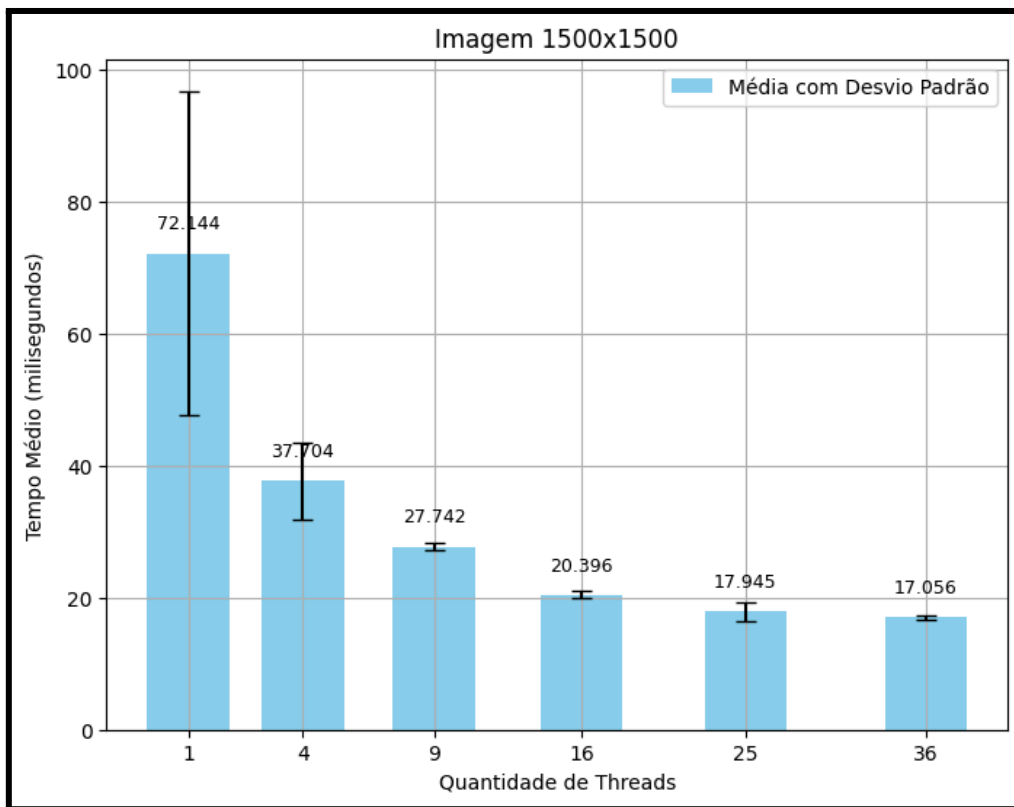


Figura 10- O mesmo gráfico que a Figura 07 só que em barras.

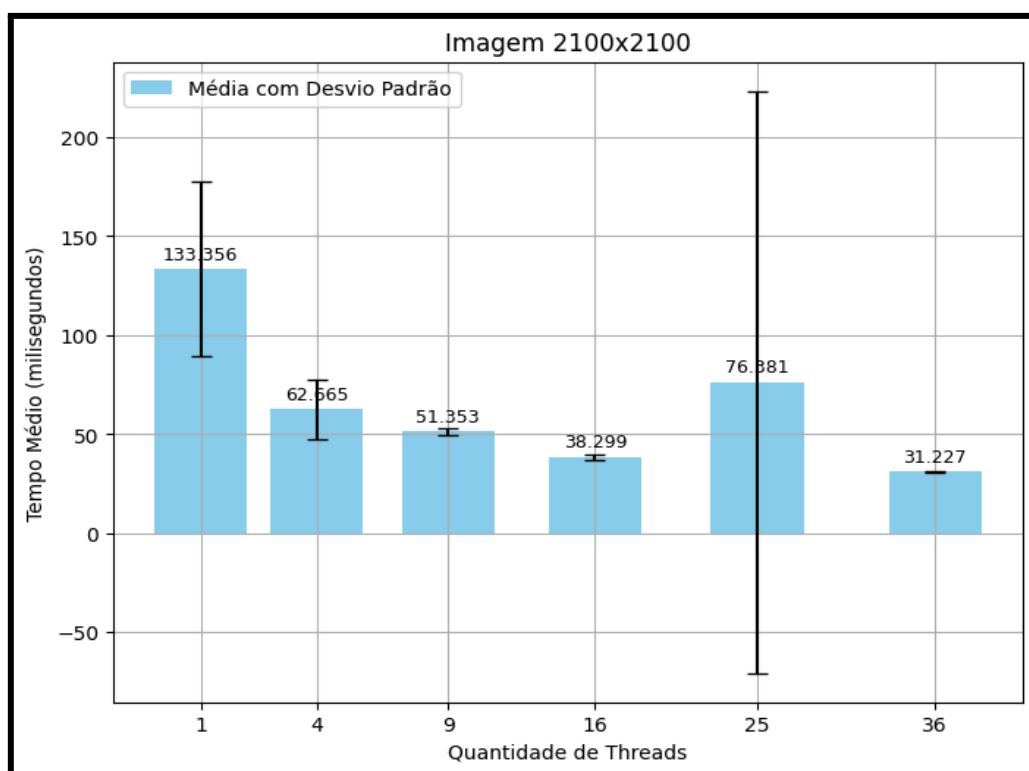


Figura 11- O mesmo gráfico que a Figura 08 só que em barras.

3)Discussão sobre a melhoria de desempenho com o uso da GPU e qual seria a diferença dessa execução em uma CPU.

A melhoria de desempenho é gigantesca. A GPU realiza todas as convoluções de cada máscara ao mesmo tempo (paralelamente) e ainda calcula a magnitude, enquanto a CPU realiza de forma sequencial (se não utilizar as threads da CPU), realizando cada convolução, uma de cada vez, e isso demonstra o quão poderoso é uma GPU para problemas envolvendo muitas operações independentes/paralelizáveis.

4)Explicação do paralelismo no algoritmo de Sobel e como ele é explorado pela GPU.

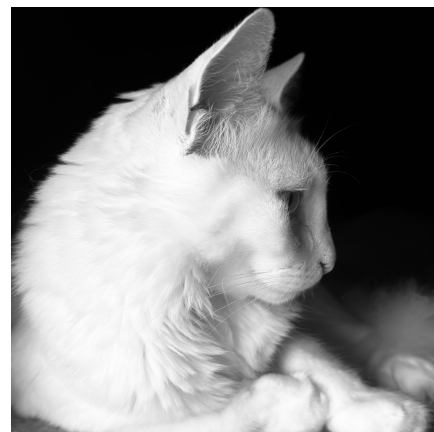
O algoritmo de Sobel por realizar as convoluções através de máscaras em uma cópia da matriz com padding e colocar o resultado do cálculo da magnitude nos índices correspondente ao centro da máscara em uma matriz de saída, torna a aplicação do paralelismo possível uma vez que não haverá dependências entre as operações realizadas em cada máscara.

O algoritmo funciona da seguinte maneira, dividimos uma imagem inteira em várias máscaras 3x3 onde cada thread fica responsável por aplicar a convolução em uma máscara (na minha implementação 1 thread cuida da máscara Gx e Gy ao mesmo tempo e ainda realiza o cálculo da magnitude, já designando o valor no pixel) e isso ocorre de forma paralela, tornando a aplicação na GPU extremamente rápida em comparação a aplicação em CPU. Em suma, a GPU realiza as operações referente a convolução de todas as máscaras simultaneamente (em vez de esperar realizar a convolução de uma máscara terminar para começar a outra).

Imagens utilizadas para teste:



Gato mago - 1024x1024



Gato - 1200x1200

Depois do filtro:



Gato mago - 1024x1024



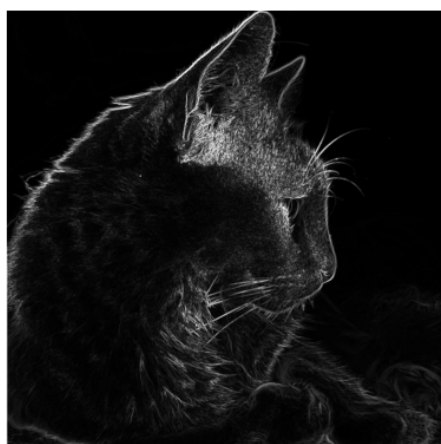
Gato mago - 600x600



Gato mago - 1500x1500



Gato mago - 2100x2100



Gato - 1200x1200



Gato - 600x600



Gato - 1500x1500



Gato - 2100x2100

Link para o ipynb:

<https://colab.research.google.com/drive/1baXEvfBaHL1FIE-7frx-FT93eWAu3uNK?usp=sharing>